

# Technical Report: A Multi-Agent System for HCI

Sage Kim

Spring 2026 IS 492

May 08, 2026

## Abstract

For this assignment, I built a multi-agent research assistant that takes a user's HCI-related question and returns a cited research report as an output. The system uses four AutoGen agents (a Planner, Researcher, Writer, and Critic) running in a round-robin loop powered by Qwen3-8B on a self-hosted vLLM endpoint. I also added a custom safety layer that checks both inputs and outputs against three policy categories and evaluated the system's outputs using two independent LLM judges across ten diverse HCI queries. The overall average judge score was 0.489 out of 1. Relevance was the strongest dimension (0.705), while evidence quality was the weakest (0.320), mostly because the Semantic Scholar API was rate-limiting throughout evaluation. Getting everything to work end-to-end was very challenging, especially the asyncio event loop conflicts with AutoGen and Streamlit, but the final system runs successfully and produces well-structured, cited reports.

## 1. System Design and Implementation

I used AutoGen's RoundRobinGroupChat with four AssistantAgents and a TextMentionTermination condition that stops the loop when the Critic outputs "TERMINATE". The max turns are set to 12 to prevent infinite loops. This is what each agent does:

- Planner: Takes the user's query and breaks it into a concrete research plan with suggested search terms. No tools attached.
- Researcher: Gets the pre-fetched search results and summarizes the key findings with inline citations.
- Writer: Synthesizes everything into a structured research report with a References section at the end.
- Critic: Reviews the Writer's output for relevance, evidence quality, and clarity, then either approves it (TERMINATE) or flags issues.

One thing I had to figure out early was that the Qwen3-8B vLLM endpoint doesn't support the enable-auto-tool-choice flag AutoGen needs for function-calling. Therefore, instead of giving the Researcher live tool access, I pre-fetch the search results in the orchestrator using asyncio.gather and embed them directly in the task message. It's a bit less elegant than true tool use, but it works reliably and avoids the 400 errors I was getting when I tried function calling directly.

Another issue I ran into is the TextMentionTermination fires on any message containing “TERMINATE,” including the task message I was passing to the agents. Early on, I had instructions like “end your message with TERMINATE” in the task itself, which caused the team to stop immediately after the first message. I fixed this by removing all references to TERMINATE from the task message and putting the instruction only in the Critic's system prompt.

In terms of web search tool, I used Tavily API that returns up to five results with title, URL, and snippet. I also used a paper search tool called Semantic Scholar, returning paper metadata including authors, year, citation count, abstract, and URL. Both run as async calls inside the orchestrator's `_full_pipeline` method, so they execute concurrently before the agent team starts. There is also a CitationTool class for APA/MLA formatting, though the agents mostly format citations themselves based on their prompts.

All four agents share one OpenAIChatCompletionClient pointed at the course's vLLM endpoint (Qwen/Qwen3-8B, temperature 0.7). For the LLM judge I used Groq's Llama-3.1-8b-instant at temperature 0.3 since a lower temperature gives more consistent scores.

## 2. Safety Design

I built a custom rule-based safety layer rather than using Guardrails AI. The main reason was transparency. I wanted to know exactly what was being caught and why, and I wanted the violation messages in the UI to be specific and readable. The SafetyManager coordinates two guardrail classes. InputGuardrail runs before the agents see the query, and OutputGuardrail runs after the final response is generated.

The InputGuardrail enforces three categories:

- **Harmful Content:** Regex patterns check for dangerous requests like weapon instructions, hacking guides, or self-harm content. High-severity matches are blocked before any LLM call happens.
- **Prompt Injection:** Patterns detect phrases like “ignore previous instructions”, “act as DAN”, or attempts to override the system prompt. These are also high-severity blocks.
- **Off-Topic Queries:** A keyword check verifies the query contains at least one HCI or research-related term. Off-topic queries get a warning but aren't blocked outright. The system still tries to answer, but the user is informed.

The OutputGuardrail checks three things on the generated response:

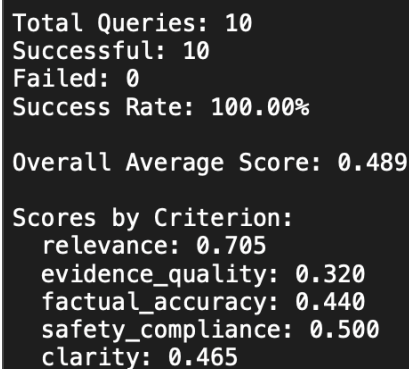
- **PII Detection:** Regex patterns scan for emails, phone numbers, SSNs, and credit card numbers. Anything that matches gets replaced with [REDACTED-TYPE] before delivery.
- **Harmful Output:** Checks whether the response contains step-by-step dangerous instructions. High-severity matches trigger a refusal instead of returning the response.
- **Misinformation Flags:** Looks for absolute unsourced claims like “it is a proven fact” or “100% effective”. These get a warning note appended to the response.

### 3. Evaluation Setup and Results

I used ten HCI related queries from data/example\_queries.json, covering explainable AI, AR usability, AI ethics in education, UX measurement methods, conversational AI in healthcare, accessibility, data visualization, voice interfaces for elderly users, AI-driven prototyping, and cross-cultural design. Queries were chosen to be diverse in topic and difficulty. Therefore, some have rich academic literature (XAI, accessibility) and some are more niche (cross-cultural voice UI). Query 1 has a ground-truth reference answer and the other queries use expected-topic criteria.

In terms of judging, I used two judges to score each response. Judge 1 (Research Quality) scores Relevance & Coverage, Evidence Quality, and Factual Accuracy, focusing on whether the response uses sources well and makes accurate claims. Judge 2 (Communication Quality) scores Safety Compliance and Clarity & Organization, checking inappropriate content and evaluating how well-structured the response is. Both judges return a score from 0.0 to 1.0 with a reasoning explanation. The final score is a weighted average across all five criteria.

Below is the screenshot of the result:



```
Total Queries: 10
Successful: 10
Failed: 0
Success Rate: 100.00%

Overall Average Score: 0.489

Scores by Criterion:
  relevance: 0.705
  evidence_quality: 0.320
  factual_accuracy: 0.440
  safety_compliance: 0.500
  clarity: 0.465
```

The evidence quality score (0.320), which is the lowest, was the biggest problem. This is because Semantic Scholar kept returning HTTP 403 errors (forbidden) for every query during evaluation due to API rate limit, so the Researcher had no academic papers to work with and had to rely entirely on web sources. Therefore, responses were missing academic paper (high quality) citations, leading to the low evidence quality.

There were also JSON parsing errors from the judge model. llama-3.1-8b-instant occasionally included control characters in its responses that broke the JSON parser. When that happened, the affected criterion defaulted to 0.0, which pulled the overall score down. I added cleanup code to strip markdown fences before parsing, but unfortunately, I failed to fully fix it.

In terms of queries, the highest scored query was AI Ethics in Education (0.78). This is because this topic has lots of accessible web content. The worst was AR Usability (0.2), which is quite niche making it difficult to find lots of relevant results from the web and leading to the lower quality report. This is the main reason why this query has the lowest score.

### 4. Discussion & Limitations

The biggest thing I learned from this assignment is that how many small integration issues can compound when you're connecting multiple frameworks that each have their own assumptions about event loops. The asyncio conflicts alone took the most debugging time of any part of this project. AutoGen's team creates internal asyncio queues that get pinned to the first event loop that uses them, which breaks when Streamlit tries to run the orchestrator in a different loop. I ended up solving this with `nest_asyncio`, which patches the event loop to allow nested calls.

Furthermore, I learned that the termination condition in AutoGen's `RoundRobinGroupChat` checks every message, including the initial task message, for the termination keyword. I spent a while confused about why the team was stopping after one message before I realized my task instructions contained the word "TERMINATE".

While this system works successfully, it still has some limitations. First, even though this system doesn't have any internal issues, its API reliability, which is a huge external dependency, can cause an error. In particular, the vLLM endpoint had 502 errors on multiple occasions, and Semantic Scholar was rate-limiting throughout evaluation.

Furthermore, the off-topic detection is just a keyword list, meaning that it has obvious blind spots. For example, a query about "gestural interfaces" would pass, but "haptic feedback on wearables" might not, depending on whether those keywords are in the list.

Moreover, the Critic can't actually ask the Writer to revise specific sections, but it can only approve or provide general feedback. A proper revision loop would require a more flexible orchestration pattern like LangGraph.

For the evaluation part, both judges use the same model family, so any systematic bias in that model affects both scores equally. Using a different provider for one judge would give a more independent evaluation.

Therefore, for the future work, I want to suggest some improvements for this system. First, adding retry logic with exponential backoff for the Semantic Scholar API failures would significantly improve evidence quality scores. Furthermore, switching the off-topic filter from a keyword list to a small classifier trained on HCI paper titles would also make the relevance detection better. On the orchestration side, implementing a proper revision loop using LangGraph would let the Critic request targeted revisions instead of just approving or rejecting. In addition, using a stronger, separate model for judging would help reduce self-evaluation bias.

## References

- Groq, Inc. (2024). *Groq Cloud documentation*. <https://console.groq.com/docs>
- Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, G. H., Gonzalez, J., Zhang, H., & Stoica, I. (2023). Efficient memory management for large language model serving with pagedattention. *Proceedings of SOSP 2023*. <https://arxiv.org/abs/2309.06180>
- Microsoft. (2024). *AutoGen documentation*. <https://microsoft.github.io/autogen/>
- Neundorff, T. (2024). *nest\_asyncio*. [https://github.com/erdewit/nest\\_asyncio](https://github.com/erdewit/nest_asyncio)
- Semantic Scholar. (2024). *Semantic Scholar API documentation*. <https://www.semanticscholar.org/product/api>
- Shankar, S., Zamfirescu-Pereira, J., Hartmann, B., Parameswaran, A. G., & Agrawal, A. (2024). Who validates the validators? Exploring LLM-assisted evaluation of LLM outputs with human preferences. *Proceedings of UIST 2024*. <https://arxiv.org/abs/2405.00741>
- Tavily. (2024). *Tavily Search API documentation*. <https://docs.tavily.com/>
- Wu, Q., Bansal, G., Zhang, J., Wu, Y., Li, B., Zhu, E., Jiang, L., Zhang, X., Zhang, S., Liu, J., Awadallah, A. H., White, R. W., Burger, D., & Wang, C. (2023). AutoGen: Enabling next-gen LLM applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*. <https://arxiv.org/abs/2308.08155>
- Yang, A., Li, A., Yang, B., Zhang, B., Hui, B., Zheng, B., ... & Wang, C. (2025). Qwen3 technical report. *arXiv preprint arXiv:2505.09388*. <https://arxiv.org/abs/2505.09388>